Spin Network Laboratory

Simulador dinámico de redes topológicas
en C++17 y SFML

Workshop 1: avance funcional

Integrantes del equipo

Dairon Steven Cabascango Erazo.

Descripción del proyecto

Spin Network Laboratory es un sistema computacional inspirado en redes de espín que permite construir, visualizar, modificar y explorar grafos dinámicos. El avance combina estructuras de datos personalizadas, administración explícita de memoria, interacción gráfica e integración numérica. Los vértices se almacenan en una tabla hash propia y sus conexiones se representan mediante listas simplemente enlazadas. Una interfaz SFML presenta la red y permite editarla mientras un modelo artificial de fuerzas organiza su geometría.

Componentes implementados actualmente en esta entrega

Lista simplemente enlazada; tabla hash FNV-1a con encadenamiento y rehash; grafo simple no dirigido; modelo de vértices y aristas; motor de fuerzas; integrador Velocity Verlet con paso fijo; controlador de interacción; renderizado y panel básico SFML; consola auxiliar; pruebas automatizadas y demo web independiente.

Enlace al repositorio de GitHub

`https://github.com/stevencabascango/Spin-Newtwork-of-Quantum-Gravity.git`

Alcance del documento

Se describe exclusivamente la versión conservada en la carpeta `Workshop 1`. La implementación de la segunda entrega se mantiene separada. Las fuerzas son herramientas de visualización: el programa no constituye una simulación completa de gravedad cuántica ni una implementación de la dinámica de redes de espín de LQG.

Descripción preparada a partir del código local y de sus pruebas. Estado de comprobación: 10 de octubre de 2026.

1. Descripción completa, propósito y objetivos

1.1. Problema que aborda el proyecto

Un grafo representa entidades y relaciones entre ellas. Cuando sus vértices o conexiones se modifican, la estructura debe conservar referencias válidas, conteos correctos y una representación coherente. Además, un dibujo estático resulta insuficiente para observar cómo cambia la geometría al conectar, mover o eliminar elementos.

El proyecto integra ambas necesidades: construye un núcleo de almacenamiento y edición con estructuras propias y lo conecta con un simulador visual. La geometría evoluciona mediante fuerzas artificiales, mientras la topología cambia únicamente por las operaciones explícitas del usuario. Mover un nodo modifica su posición, pero no crea ni elimina conexiones.

1.2. Objetivo general

Desarrollar un avance ejecutable en C++17 capaz de almacenar, editar y visualizar un grafo no dirigido, utilizando una tabla hash y listas enlazadas propias, y actualizar sus posiciones mediante integración numérica de paso fijo.

1.3. Objetivos específicos

1. Implementar una lista simple con inserción, búsqueda, eliminación, limpieza e iteración.
2. Implementar una tabla hash con FNV-1a, colisiones por encadenamiento y redimensionamiento controlado.
3. Representar vértices por identificadores y conexiones recíprocas sin punteros colgantes entre vértices.
4. Mantener las reglas de grafo simple: sin lazos, sin aristas duplicadas y sin extremos inexistentes.
5. Incorporar atracción elástica, repulsión regularizada y amortiguamiento.
6. Separar el paso físico del ritmo de renderizado mediante Velocity Verlet y un acumulador temporal.
7. Permitir selección, arrastre, fijación, creación, conexión, eliminación y pausa en SFML.
8. Comprobar estructuras, dinámica e interacción con pruebas reproducibles.

1.4. Interpretación del modelo

El término `QuantumVertex` pertenece al lenguaje conceptual del proyecto: computacionalmente es un vértice etiquetado con estado físico artificial. El espín de una arista es una etiqueta semientera positiva. No se imponen reglas de acoplamiento SU(2), amplitudes cuánticas, restricciones gravitacionales ni una correspondencia con la dinámica física de la gravedad cuántica.

La contribución de esta entrega es computacional y académica: mostrar que estructuras personalizadas pueden sostener un grafo editable y una visualización dinámica. La validación de ese software es distinta de demostrar una teoría física.

Base de implementación: `README.md`, `QuantumGraph.hpp` y `SimulationController.hpp`.

2. Arquitectura, objetos y administración de memoria

2.1. Organización por responsabilidades

```
DesktopApplication -> InputController / GraphRenderer / HUD
        |
        +-- SimulationController
               +-- QuantumGraph
               |      +-- HashTable<uint64_t, QuantumVertex>
               |                    +-- QuantumVertex
               |                          +-- SinglyLinkedList<SpinEdge>
               +-- ForceEngine / VerletIntegrator / FixedStepClock
```

El núcleo y la lógica de interacción no necesitan abrir una ventana para ejecutarse en las pruebas. SFML se utiliza en la capa de presentación: ventana, eventos, dibujo y texto. Esta separación permite evaluar fallos de estructuras o física independientemente del sistema gráfico.

2.2. Vértices y aristas

Objeto	Información almacenada
`QuantumVertex`	ID; masa y volumen; posición actual y anterior; velocidad, aceleración y fuerza; estado `pinned`; lista simple de conexiones.
`SpinEdge`	ID lógico de arista, `targetId`, espín, longitud natural y rigidez.
`Vec2`	Dos componentes reales y operaciones vectoriales elementales.
`PhysicsParameters`	Rigidez global, repulsión, amortiguamiento, regularización y paso físico fijo.

El volumen se conserva como atributo del vértice; no se utiliza para calcular las fuerzas de esta versión. La dinámica usa masa, posiciones, velocidades y parámetros de las conexiones. Esto evita atribuir a un campo almacenado una función que todavía no tiene.

2.3. Propiedad de los recursos

La tabla posee los vértices. Cada vértice posee su lista de adyacencias. Los nodos de hash y lista poseen el nodo siguiente mediante `std::unique_ptr`; el puntero de cola es observador. No se usan `new` y `delete` manuales para administrar esos nodos.

Las aristas contienen IDs de destino y no punteros directos al otro vértice. Por ello, un rehash puede reconstruir el almacenamiento sin alterar el significado de las conexiones. Los punteros devueltos por `findVertex` son observadores temporales y no deben conservarse tras rehash, eliminación o asignación.

2.4. Copia, movimiento y limpieza

Las listas y tablas admiten copia profunda: copiar un grafo duplica sus vértices y listas internas. El movimiento permite transferir estructuras y reutilizar el origen. La destrucción de las cadenas es iterativa para evitar una recursión proporcional al número de nodos. Estas capacidades preparan el historial de la segunda entrega, pero no significan que el historial C++ esté incluido en Workshop 1.

Base de implementación: `include/Core/`, `include/DataStructures/` y `src/Graphics/`.

3. Lista simplemente enlazada personalizada

3.1. Estructura interna

`SinglyLinkedList<T>` contiene una cabeza propietaria, una cola observadora y un contador. Cada nodo almacena un valor de tipo `T` y un `unique_ptr` al siguiente. La lista se usa para las adyacencias de cada vértice, por lo que el costo de recorrerla depende de su grado.

```
head -> [valor | next] -> [valor | next] -> [valor | null]
                                                  ^
                                                tail (observador)
```

3.2. Operaciones implementadas

Operación	Comportamiento	Costo
`pushFront`	Inserta al inicio; establece cola cuando estaba vacía.	$O(1)$
`pushBack`	Inserta al final sin recorrer toda la cadena.	$O(1)$
`find`, `contains`	Buscan por igualdad; la búsqueda admite acceso constante o mutable.	$O(n)$
`remove`	Elimina la primera coincidencia por valor.	$O(n)$
`removeIf`	Elimina por predicado; útil para buscar una arista por destino.	$O(n)$
`clear`	Desmonta la cadena y deja cabeza/cola vacías.	$O(n)$
`size`, `empty`	Consultan el contador y el estado vacío.	$O(1)$
`begin`, `end`	Permiten iteración sin exponer los nodos propietarios.	Inicio $O(1)$

3.3. Eliminación correcta de cabeza y cola

La eliminación opera sobre el enlace propietario que apunta al nodo encontrado. Ese enlace puede ser la cabeza o el `next` del nodo anterior. El siguiente nodo ocupa el enlace del retirado. Si se elimina la cola, el observador pasa al anterior; si era el único elemento, queda nulo. El contador se reduce una vez por eliminación efectiva.

3.4. Copias independientes y pruebas

La copia reconstruye una nueva cadena con los valores del origen, evitando compartir nodos internos. La asignación usa un temporal e intercambio. El movimiento deja el origen vacío y permite insertar nuevos valores en él.

Las pruebas comprueban inserción por ambos extremos, búsquedas existentes y ausentes, eliminación de cabeza y cola, reinserción después de vaciar, iteración constante y copia/movimiento. Se construye también una lista de 10 000 elementos para comprobar que una copia permanece independiente y que la limpieza funciona sin recurrir a liberaciones manuales.

Base de implementación: `SinglyLinkedList.hpp` y `tests/test_core.cpp`.

4. Tabla hash: FNV-1a, colisiones y rehash

4.1. Almacenamiento por identificador

`HashTable<uint64_t, QuantumVertex>` relaciona el ID con el vértice. Un arreglo de buckets inicia cadenas propias de nodos clave–valor. Una búsqueda calcula el bucket y recorre su cadena; no utiliza `std::unordered_map` como sustituto de la tabla evaluada.

4.2. Función hash implementada

Para bytes b_i de la clave, FNV-1a de 64 bits se calcula como

$$h_0 = 14695981039346656037, \qquad h_{i+1} = (h_i \,\mathrm{XOR}\, b_i)\, 1099511628211 \pmod{2^{64}}.$$

El índice es $h(k)$ mód m, donde m es el número de buckets. Las claves enteras se codifican en bytes little-endian para obtener un resultado independiente del orden de bytes del procesador. La tabla también admite cadenas y un hash alternativo inyectable para las pruebas.

4.3. Resolución de colisiones

Dos claves con el mismo índice se guardan en la misma cadena, pero siguen distinguiéndose por su valor de clave. Insertar un ID repetido devuelve rechazo y no sobrescribe silenciosamente el vértice. Eliminar una entrada repara el enlace de la cadena.

```
bucket[0] -> [clave A, valor] -> [clave B, valor] -> null
bucket[1] -> null
bucket[2] -> [clave C, valor] -> null
```

4.4. Factor de carga y redimensionamiento

El factor de carga es $\alpha = n/m$. Antes de una inserción que supere 0.75, la tabla solicita una capacidad mayor. El rehash reconstruye una tabla temporal y la intercambia con el original únicamente después de completar las inserciones. Copiar valores durante ese proceso incluye copiar las adyacencias del vértice.

La capacidad finalmente elegida puede superar una solicitud demasiado pequeña para respetar el factor de carga. Los observadores de vértices pueden invalidarse: el código no debe depender de conservar sus direcciones durante un rehash.

4.5. API, costos y evidencia

La API incluye `insert`, `remove`, `find`, `contains`, `clear`, `size`, `capacity`, `loadFactor` y `rehash`. El costo esperado de consulta es $O(1)$ con distribución adecuada; con colisiones patológicas puede llegar a $O(n)$.

Las pruebas insertan y buscan 5 000 vértices, rechazan duplicados, reconstruyen a 16 384 buckets y eliminan todas las entradas. Un hash constante fuerza 1 000 claves al mismo bucket y verifica consultas y eliminaciones. También se comprueba FNV-1a de `hello` contra el valor esperado `0xa430d84680aabd0b`.

Base de implementación: `HashTable.hpp` y `tests/test_core.cpp`.

5. Grafo no dirigido y componentes del núcleo

5.1. Representación y contratos

El grafo es $G = (V, E)$, simple y no dirigido. Los vértices están indexados en la hash y cada uno posee su lista de adyacencias. Crear una conexión entre a y b inserta los registros $a \to b$ y $b \to a$ con el mismo ID lógico. El contador de aristas aumenta una vez, no dos.

Se rechazan lazos ($a = b$), duplicados y extremos inexistentes. Masa no positiva, posiciones no finitas y parámetros de arista inválidos producen rechazo o excepción según la operación. El espín debe ser positivo y semientero; la longitud natural, positiva; la rigidez, no negativa.

5.2. Operaciones implementadas

Operación	Función
`addVertex`, `removeVertex`	Crear o eliminar vértices e incidentes.
`addEdge`, `removeEdge`	Mantener las dos adyacencias y el conteo lógico.
`findVertex`, `containsVertex`	Localizar vértices por ID.
`vertexCount`, `numberOfEdges`	Consultar tamaños del grafo.
`degree`, `neighbors`	Consultar conexiones de un vértice.
`ids`, `forEachVertex`	Recorrer y obtener IDs ordenados.
`clear`, `rehash`	Limpiar o reorganizar almacenamiento.

Se conservan también nombres cortos de la API original: `size`, `edgeCount` y `contains`. El código externo no debe modificar directamente las adyacencias: las operaciones del grafo mantienen su reciprocidad.

5.3. Eliminación y consistencia

Para eliminar un vértice se obtienen primero sus vecinos. Se retira cada arista en ambos extremos, se reduce el contador y finalmente se elimina el vértice de la hash. Así ningún vecino conserva una conexión al ID eliminado. Si falla la creación de la segunda adyacencia de una arista, se retira la primera antes de propagar el error.

5.4. Métricas complementarias ya disponibles

El núcleo conserva funciones básicas de análisis y una consola: número de vértices/aristas, grado, componentes conexas y ciclos independientes. Si C es el número de componentes conexas, para un grafo 1D se usan $\beta_0 = C$, $\beta_1 = E - V + C$ y $\chi = V - E$. Estas funciones son un complemento del avance; no equivalen al panel analítico, lista doble e historial C++ de la segunda entrega.

5.5. Prueba representativa

Un grafo de 1 000 vértices y 5 000 aristas tiene grado 10 en cada nodo del caso probado. Al eliminar el vértice 0 se retiran sus 10 conexiones y el conteo baja a 4 990. Su copia mantiene el grafo original; reinsertar el ID eliminado crea un vértice aislado.

Base de implementación: `QuantumGraph.hpp`, `GraphMetrics.hpp` y `tests/test_core.cpp`.

6. Dinámica artificial y parámetros

6.1. Modelo de visualización

Las fuerzas buscan una distribución visual comprensible: resortes para las conexiones, repulsión entre pares y amortiguamiento para reducir oscilaciones. No modifican las conexiones. Las coordenadas y parámetros utilizan unidades internas de simulación; no se presentan como mediciones físicas en unidades SI.

Para posiciones $\mathbf{x}_i$, se define $\boldsymbol{\Delta}_{ij} = \mathbf{x}_j - \mathbf{x}_i$ y $r_{ij} = \|\boldsymbol{\Delta}_{ij}\|$. La fuerza del resorte implementada sobre i es

$$\mathbf{F}_{ij}^{s} = k_{\text{global}} k_{ij} (r_{ij} - L_{ij}) \frac{\boldsymbol{\Delta}_{ij}}{r_{ij} + \varepsilon_s}.$$

Cuando $r_{ij} > L_{ij}$ atrae hacia j; cuando es menor, separa. Cada arista lógica se evalúa una vez y contribuye con signos opuestos a sus extremos.

6.2. Repulsión regularizada

Para un par distinto,

$$\mathbf{F}_{ij}^{r} = -\frac{k_r}{r_{ij}^2 + \varepsilon^2} \widehat{\boldsymbol{\Delta}}_{ij}.$$

La regularización usa $\varepsilon = 5$ en el valor inicial. Si dos nodos coinciden, se elige una dirección determinista basada en los IDs; esto produce separación sin dividir por cero. La interacción se evalúa para todos los pares, de costo $O(V^2)$. El espín es una etiqueta: no modifica directamente las fuerzas de este avance.

6.3. Amortiguamiento y masa

Se utiliza $\mathbf{F}_i^d = -\gamma \mathbf{v}_i$ y $\mathbf{a}_i = \mathbf{F}_i / m_i$. Las fuerzas conservativas se acumulan en `ForceEngine`; el amortiguamiento se aplica por separado en el integrador mediante una exponencial que incluye la masa. Un nodo fijo no recibe masa infinita: `pinned=true` impide avanzar su posición y mantiene velocidad cero.

Parámetro inicial	Valor	Papel
Rigidez global	2.0	Escala los resortes de aristas.
Repulsión	12 000	Evita agrupación excesiva.
Amortiguamiento	2.0	Reduce velocidades y oscilaciones.
Regularización	5.0	Acota repulsión a distancias pequeñas.
Longitud de los presets	90	Longitud natural por arista.
Masa de presets	1.0	Respuesta inercial artificial.

Para una interpretación dimensional interna, rigidez efectiva tiene dimensión M/T^2, repulsión ML^3/T^2 y amortiguamiento M/T. Los números son parámetros del modelo gráfico, no constantes gravitacionales.

Base de implementación: `ForceEngine.hpp`, `PhysicsParameters.hpp` y `QuantumVertex.hpp`.

7. Velocity Verlet y actualización de paso fijo

7.1. Algoritmo conservativo

Velocity Verlet divide el impulso de velocidad en dos medios pasos. Primero se calculan fuerzas en la posición actual; luego se avanza la posición y se recalculan fuerzas:

$$\begin{aligned}\mathbf{v}_{n+1/2} &= \mathbf{v}_n + \tfrac{\Delta t}{2}\mathbf{a}_n,\\ \mathbf{x}_{n+1} &= \mathbf{x}_n + \Delta t\,\mathbf{v}_{n+1/2},\\ \mathbf{a}_{n+1} &= \mathbf{F}(\mathbf{x}_{n+1})/m,\\ \mathbf{v}_{n+1} &= \mathbf{v}_{n+1/2} + \tfrac{\Delta t}{2}\mathbf{a}_{n+1}.\end{aligned}$$

La versión implementada incorpora amortiguamiento con separación simétrica de operadores. Para $d = \exp[-\gamma\Delta t/(2m)]$, realiza

$$\mathbf{v}_{1/2} = d\mathbf{v}_n + \tfrac{\Delta t}{2}\mathbf{a}_n, \qquad \mathbf{v}_{n+1} = d\big(\mathbf{v}_{1/2} + \tfrac{\Delta t}{2}\mathbf{a}_{n+1}\big).$$

La posición se avanza con $\mathbf{v}_{1/2}$. Esto evita tratar el amortiguamiento dependiente de velocidad como una fuerza dependiente únicamente de posición.

7.2. Reloj físico independiente de FPS

El paso fijo es $\Delta t = 1/120$ s. `FixedStepClock` acumula el tiempo transcurrido entre frames y ejecuta pasos completos. A 60 FPS, normalmente se realizan dos pasos físicos por frame; no se utiliza directamente el tiempo variable del frame como paso de integración.

Pausar reinicia el acumulador para no recuperar el tiempo de pausa. Una suspensión larga se limita a 0.1 s de recuperación, como máximo 12 pasos. Esa política descarta tiempo deliberadamente tras bloqueos prolongados; la equivalencia entre FPS se comprueba en condiciones sin esos bloqueos.

7.3. Fijación, arrastre y comprobaciones

Un vértice fijo o arrastrado mantiene su posición durante la física y tiene velocidad cero. El arrastre conserva el desplazamiento entre el centro del nodo y el punto donde se agarró, evitando un salto inicial. La posición anterior y aceleración se actualizan como parte del estado numérico.

Las pruebas comparan el resultado tras un segundo a 30, 60 y 144 FPS con tolerancia 10^{-8} en posiciones. Un oscilador sin amortiguamiento mantiene error relativo de energía inferior a 0.1 % tras 1 200 pasos; la prueba amortiguada confirma disipación. Una masa libre cumple el decaimiento esperado $v(t) = v(0)e^{-\gamma t/m}$. Estas son comprobaciones de los casos ejecutados, no una garantía universal de estabilidad para cualquier parámetro o grafo.

Base de implementación: `VerletIntegrator.hpp` y `tests/test_physics.cpp`.

8. Interfaz SFML, controles y demo complementaria

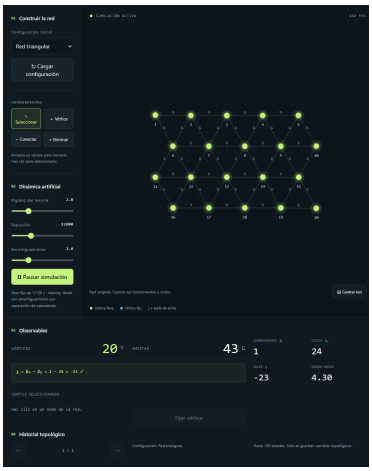
8.1. Presentación implementada

`GraphRenderer` dibuja aristas como segmentos, vértices como círculos e IDs junto a los nodos. `HUD` presenta herramientas, conteos, estado de pausa, parámetros y FPS. `InputController` traduce eventos y coordenadas de pantalla a acciones sobre el grafo; la cámara permite encuadrar sin alterar la geometría.

Acción	Control del Workshop 1
Seleccionar / arrastrar	Mouse izquierdo; herramienta Seleccionar o `S`.
Agregar vértice	Botón Agregar y clic; `A` en el lienzo.
Crear arista	`C` o herramienta Conectar; origen y destino.
Eliminar seleccionado / bajo puntero	`Delete` / `D` y clic.
Pausa / fijación / posiciones	`Space` / `P` / `R`.
Repulsión / rigidez / amortiguamiento	Flechas arriba/abajo; corchetes; coma/punto.
Red inicial / red de 100 nodos	`1` / `2`.
Encuadrar / cerrar	`F` / `Esc`.

La red inicial tiene 20 nodos y 43 aristas; la configuración de 100 nodos tiene 261 aristas. El límite de interacción visual es 500 vértices, independiente de la capacidad de la tabla hash. Reiniciar posiciones conserva topología y fijaciones.

8.2. Demo web independiente



Captura real de la demo web del avance en Brave; no es una captura SFML.

La página `site/dist/index.html` es un programa JavaScript independiente y puede abrirse sin servidor. Muestra edición, fuerzas y métricas; su historial web es complementario y no acredita una lista doble o historial C++ del Workshop 1. Sus cambios se pierden al recargar.

La aplicación nativa está implementada y compilada, pero la pantalla X11 fue bloqueada en el entorno restringido. Por tanto, el documento distingue controles probados en el controlador de una validación visual completa de la ventana SFML, que sigue pendiente.

Base de implementación: `src/Graphics/`, `SimulationController.hpp` y captura local de la demo web.

9. Pruebas, resultados y complejidades

9.1. Evidencia del avance

El Workshop 1 tiene tres suites: `test_core`, `test_physics` y `test_interaction`. Sus binarios se ejecutaron nuevamente el 10 de octubre de 2026 y finalizaron correctamente. Los resultados se respaldan en las aserciones de sus fuentes; no se sustituyen por una inspección visual del código.

Caso comprobado	Resultado
Hash propia	Inserción/búsqueda de 5 000 vértices; duplicados rechazados; rehash y eliminación completa.
Colisiones	1 000 claves forzadas a un bucket; búsqueda y borrado de extremos/interior.
Lista simple	Operaciones, copia/movimiento y cadena de 10 000 elementos.
Grafo	1 000 vértices, 5 000 aristas y retiro de conexiones incidentes.
Estabilidad numérica	100 nodos y 261 aristas durante 7 200 pasos: 60 s simulados, estados finitos.
Independencia de FPS	Estados coincidentes en el caso probado a 30/60/144 FPS.
Interacción	Creación, conexión, rechazo de lazos/duplicados, arrastre, fijación, pausa y eliminación.

9.2. Costos de los algoritmos

Operación	Costo / alcance
Lista: insertar al frente/final	$O(1)$. Búsqueda/eliminación: $O(n)$.
Hash: consulta	$O(1)$ esperado; $O(V)$ en cadenas patológicas.
Rehash / copia de grafo	$O(V+E)$ esperado; colisiones patológicas pueden encarecer reconstrucción.
Fuerzas / paso Verlet	$O(V^2+E)$ esperado; dos evaluaciones de fuerzas.
Dibujo	$O(V+E)$ esperado.
IDs ordenados	$O(V \log V)$; auxiliar para reinicio y recorridos.

Estos costos suponen cadenas hash razonablemente distribuidas. Copiar un vértice incluye sus adyacencias. La estabilidad de 100 nodos no autoriza prometer rendimiento para millones de elementos.

9.3. Comprobaciones pendientes

Los registros del avance indican ASan/UBSan satisfactorios con `detect_leaks=0`. LeakSanitizer no funcionó bajo ptrace: no se afirma ausencia de fugas verificada. Tampoco se comprobó la ventana SFML en el sandbox ni la compilación Windows/Code::Blocks. La prueba `--smoke-test` y el guion manual están preparados para un escritorio habilitado.

Base de implementación: `tests/test_core.cpp`, `test_physics.cpp`, `test_interaction.cpp` y `docs/validation.md`.

10. Archivos, ejecución y enlace al repositorio

10.1. Entrega del Workshop 1

```
Workshop 1/
  main.cpp                  SpinNetworkLaboratory.cbp
  CMakeLists.txt            README.md / ENTREGA.txt
  include/                  src/
  tests/                    assets/fonts/
  docs/architecture.md      docs/validation.md
  docs/report/              site/dist/
  tools/                    bin/ / .deps/sfml/
```

La carpeta conserva el avance de manera independiente. Los fuentes, recursos, configuración de compilación y pruebas están incluidos. El SDK local y binarios suministrados son para Linux x86_64; no son ejecutables de Windows.

10.2. Compilación y ejecución reproducibles

Desde la carpeta del Workshop 1, con GCC C++17 y SFML 2.6 compatibles:

```
./tools/build.sh
./bin/spin_network
./tools/build.sh --core-only
./bin/spin_network --smoke-test
```

El modo `--core-only` prueba estructuras, física e interacción sin SFML. Con CMake se usan `cmake -S . -B build`, `cmake --build build` y `ctest --test-dir build --output-on-failure`. Si se utiliza el SDK local, debe indicarse su prefijo en la configuración.

En Windows se abre `SpinNetworkLaboratory.cbp` en Code::Blocks, se configura un SDK SFML 2.6 para MinGW compatible con el compilador, se enlazan graphics/window/system y se colocan las DLL y recursos junto al programa. Las instrucciones de integración se apoyan en la documentación oficial de SFML [1].

10.3. Repositorio proporcionado

https://github.com/stevencabascango/Spin-Newtwork-of-Quantum-Gravity.git

El enlace se reproduce exactamente como fue facilitado por el integrante. El contenido descrito y las pruebas corresponden a la copia local del Workshop 1; este documento no certifica que el estado remoto sea idéntico. La segunda entrega agrupa lista doble, historial C++, undo/redo, panel analítico final y refinamiento; esas funcionalidades no se atribuyen al avance descrito aquí.

10.4. Referencias y preparación del documento

Referencias

[1] SFML. *SFML and Linux*, documentación oficial, versión 2.6. https://www.sfml-dev.org/tutorials/2.6/getting-started/linux/. Apoya la configuración de compilación y enlazado.

[2] Código local de *Spin Network Laboratory, Workshop 1*: `include/`, `src/`, `tests/`, `README.md` y `docs/validation.md`. Base de la descripción y resultados.

[3] Kassis, T.; Agarwal, V.; He, Y.; Patel, D.; Brueckner, A. M. (2026). *Scientific Agent Skills: A Library of Procedural Knowledge for Research Agents*. https://doi.org/10.48550/arXiv.2609.00065. Herramienta metodológica de preparación asistida; no evidencia de los resultados del simulador.